

Lab 11

Host discovery (2 points per row; 10 total points)

Host Machines	Open Ports/Services	Operating System
Ambulance Laptop (X.X.X.158)	135/TCP(msrpc) 139/TCP(netbios-ssn) 445/TCP(Microsoft-DS Active Directory,SMB), 3389/TCP (ms-wbt-server(Windows Remote Desktop))	The Operating system is Microsoft Windows, however there is a possibility that the exact version could either be Server 2008 R2, Vista SP2, NetWare 6 or Windows 8/8.1 Update 1
Database (X.X.X.156)	None	This system is running Ubuntu Linux 18.04 LTS with the version of 4.15.0-213-generic for x86_64 architecture.
Web Server (X.X.X.150)	22/TCP(SSH) 44245(Telnet) 8000(http)	Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Clinician Desktop (X.X.X.154)	135/TCP(msrpc) 139/TCP(netbios-ssn) 445/TCP(Microsoft DNS)	The Operating System over here, was identified as Microsoft Windows The most likely versions that could be used over here are one of these ones: Windows XP SP2 or SP3, Windows Embedded Standard 2009, Windows Server 2003.
Reception Desktop (X.X.X.152)	135/TCP(msrpc) 139/TCP(netbios-ssn) 445/TCP(Microsoft-DS Active Directory,SMB), 3389/TCP (ms-wbt-server(Windows Remote Desktop)) 49666 (Unknown) 49667 (Unknown)	The Operating system is Microsoft Windows, however there is a possibility that the exact version could either be Server 2008 R2, Vista SP2, NetWare 6 or Windows 8/8.1 Update 1

Exploiting three machines (10 points per row; 30 total points)

Host Machines	How did you gain access?	What <u>specific</u> harm could be done?	How can you remediate it?
Ambulance Laptop	Over here, I basically compromised the Reception desktop first and then I was able to dump the NTLM password hashes that were basically stored in the machine's memory. So,	After I got the access, I was basically able to view all and open any sensitive paramedic incident reports that were stored on the machine. The reports basically contained	Since the compromise happened because the credential hashes were easy to extract and reuse and there were no major authentication requirements to view the sensitive

	I took a password hash for one of the users, Tom who had access to the machine. Using that credential I basically performed a pass the hash attack through SMB, which then allowed me to authenticate to the machine without even knowing its password since the SMB authentication accepted Tom's NTLM hash.	information about emergency calls, medical conditions and also some information about people who contacted the hospital in the past couple of days. If an attacker gets hold of this information then he can basically expose any confidential details of the patients and also blackmail them to make their sensitive medical report public if they didn't give him the ransom that he demands.	information, the hospital can prevent these types of attacks from occurring by limiting the administrative privileges on the Reception desktop so that the credentials cannot be dumped anyone easily, disable LM, NTLM and SMB authentication methods with some stronger authentication methods such as NTLMv2 and kerberos so that we can reduce the attacking surface and also make the pass the hash attacks ineffective, to make it difficult for the attacker to hack any information.
Database	With physical access to the database, I basically booted a kali image and then mounted the database's harddrive. Once I mounted that, I was freely able to browse the any filesystems in the machine and I was able to access the MySQL data directory and that allowed me to basically read and modify any information that was stored in the files.	I was able to open the customers.ibd in the database which basically contains a list of people along with their date of birth, card numbers and SSNs, which an attacker could steal quite easily and sell it on dark web or cause an identity theft.	The hospital must enable a full disk encryption on the server which would prevent the drive from being mounted without the encryption key or password, since it did not ask me for it. Secondly, the database itself must use data-at-rest encryption so that the important can only be viewed or read by someone, which would not allow an attacker to edit the file or cause any harm to the hospital.
Web Server	First I tried to gain the root access to the server by initiating an SSH connection directly to its IP address. Since the ssh service was misconfigured over here, so it allowed us to login without the requiring any password or key authentication. Then later on, under the /var/www/html I found a	The /etc/shadow and the secureCrypt files that we just found out could create major security risks for the hospital as an attacker can crack hashed passwords offline and recover any weak or reused passwords without triggering any kind of alert. The cracked passwords can then be used to log in to the other	In order to remediate any issues discovered, the hospital must reset all the passwords for their staff, administrators and system accounts and implement any strong password policies that require long and complex passphrases. Also, the web servers should be rebuilt from a clean or a trusted image rather than a

	hidden directory, .hacked, which contained a copy of the system's /etc/shadow file where I could all the local account password hashes had exposed. Also, under the /var/www/html/flaskApp I found secureCrypt.py, which was used for "encrypting" the user passwords. When I read the file, I found out that it basically performs a simple Caesar cipher on the reversed text and then adds some random characters in some fixed positions which could be guessed quite easily and moreover it uses a hard-coded symmetric key too. Also, any encrypted password stored by the application can be decrypted immediately since the algorithm is fully reversible and the key is embedded in the code too.	systems which includes staff computers, internal servers and the Ambulance laptop which gives an attacker a broader access across the network. Also, our other vulnerable file, secureCrypt.py's encryption could be reversed easily will allow an attacker to decrypt any sensitive data that is stored or transmitted through the web applications including any usernames, staff passwords, patient identifier, and any administrator credentials. Now, since the attacker has access to both /etc/shadow and secureCrypt.py, the attacker can now just impersonate the staff, modify or steal any patient records, disrupt any medical operations, escalate privileges and maintain unauthorized access for a long period of time.	patched one because to ensure that all the backdoor or malicious files are removed since the attacker has obtained the root level access. The secureCrypt.py must be removed and replaced with any cryptographic methods or a trusted library. The hospital must also perform a full network audit so that we can detect any further unauthorized access, persistent mechanisms or any kind of account misuse across any devices. Also, they must implement any proper server hardening file permission controls, intrusion detection and regular security monitoring further reduces the risk of having any similar compromises in the future.
--	---	---	--

Sensitive information (10 points per row; 30 total points)

Host Machines	What information I found, and why it's bad that I can see it.
Ambulance Laptop	On this machine, I was basically able to access a list of any paramedic incident reports which contained sensitive information such as patient names, paramedic names, incident descriptions, dates etc. It is highly confidential data that must be protected since the fact that I was able to view this information implies that anyone can do it and compromise the machine which could lead to exposure of some personal medical details of patients which will not only violate their privacy, but also make them a potential target in the attacker's mind. It also mentions about some operational details about some emergency responses, which could be easily exploited by an attacker.
Database	When I went through the directories/files on the mounted database, I found customers.ibd, which contains many sensitive records such as the customer's username along with their SSN number, date of birth, and credit card numbers. An

	attacker can easily steal this information and since the hospital hasn't set up authentication methods yet. It could also lead to identity thefts and frauds.
Web Server	When I was investigating, I discovered two critical pieces of information: a copy of the /etc/shadow file, which contains hashes for all local accounts and administrators, and the source code for the hospital's custom encryption script. Being able to see the hashes in the /etc/shadow file is very dangerous because an attacker can download the file and crack weak or reused passwords offline without any detection, allowing them to impersonate staff or access other machines in the network. Moreover, the secureCrypt.py exposes exactly how the web application "encrypt" sensitive data. Since the algorithm is very weak and fully reversible, an attacker who gains access to this script can basically decrypt any stored credentials or any patient information as well. Since we can view both of these files, means that the system has been compromised at a very deep level and the attacker can steal passwords, decrypt any sensitive data, and move freely through the hospital network.

Remediation (10 points per row; **30 total points**)

Host Machines	Vulnerabilities, misconfigurations, sensitive information disclosures, malpractices	Does the issue need to be fixed? Why, or why not?	If actions were taken, how did you remediate it
Ambulance Laptop	The machine basically stores the paramedic reports in unencrypted format which could be easily exploited by the attackers. This machine points to the fact that the authentication used for it are very poor and give some overly permissive administrative to anyone since I was able to access the machine by using the credential hashes that I dumped for performing the pass the hash attack. Additionally, the SMB service also gets exposed over here since it allowed me to get an unauthorized access to the to any confidential medical information.	Yes, this issue needs to be fixed since the machine stores some very sensitive information about the hospital's operational services and their patients' medical records as well and it may also violate a patient's privacy since anyone can read this without getting any proper authentication, plus it also puts them and the paramedics at the risk of getting blackmailed and misuse of their medical information.	In order to remediate this issue, I would first reduce any administrative privileges on the reception desktop so that the hashes cannot be stolen easily by the attacker. Secondly, I would enforce the usage of some stronger security methods such as NTLMv2, so that we can prevent any pass the hash attacks in the future and lastly I would basically try to encrypt the entire so that read even if they are not accessed through the proper channel.
Database	In this machine, basically the server's hardware was	Yes, this issue needs to be fixed because any person	I did not modify or repair anything on the actual

	not encrypted, which actually allowed me to mount it using a live Kali environment and directly access any kind of raw database files. Some sensitive data like SSN numbers, credit card number, date of birth etc. were also stored in a plaintext file, customers.ibd, which is another major security vulnerability.	with root level privileges can basically read, write, edit and steal personal information from important since any authentication hasn't been set up as of now.	system, but still by observing the files on the machine I can say that an appropriate way to remediate would be to enable full disk description on the webserver, implement proper database level encryption for any sensitive fields and restrict any physical access to the machine, so that only authorized people are allowed to login and access the information.
Web Server	Over here, the server contained some severe vulnerabilities like having a copy of the /etc/shadow file in a hidden directory, .hacked, which proves that an attacker gained all the root access and has exfiltrated all the password hashes in the system. These hashes could be cracked offline to obtain any staff or administrator passwords. The file secureCrypt.py has also been discovered over here, which allows the attacker to decrypt any sensitive data stored by the web application. Moreover, the file permissions were misconfigured and the presence of any directories that were created by the attacker indicates that the system has been compromised.	Yes, the issue needs to be fixed because if the attacker gains the root access then they can basically steal the passwords and decrypt any sensitive information.	The hospital must reset all the passwords for their local system, staff and administration accounts, remove any directories found that were created by the attacker and then completely rebuilt the web server since the attacker had attained root level privileges